```
import os
import zipfile
import requests
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow.keras import layers, models
from tensorflow.keras.applications import MobileNetV2
from sklearn.metrics import classification_report, confusion_matrix
from google.colab import files
from tensorflow.keras.utils import load_img, img_to_array

print("TensorFlow Version:", tf.__version__)

print("\nDownloading dataset...")

url = "https://github.com/vebynovalisa/fruit-freshness-
classification/raw/refs/heads/main/Dataset.zip"

zip_path = "/content/Dataset.zip"

response = requests.get(url)

with open(zip_path, "wb") as f:
    f.write(response.content)

print("Dataset downloaded successfully")

extract_path = "/content/fruit_dataset"

with zipfile.ZipFile(zip_path, "r") as zip_ref:
    zip_ref.extractall(extract_path)

print("Dataset extracted successfully")

print("\nChecking dataset folders...")

for root, dirs, file_list in os.walk(extract_path):
    if file_list:
        print(root, ":", len(file_list), "images")


def find_folder(base_path, folder_name):
    for root, dirs, file_list in os.walk(base_path):
        if os.path.basename(root).lower() == folder_name.lower():
            return root
    return None


train_path = find_folder(extract_path, "train")
```

```
valid_path = find_folder(extract_path, "valid")
test_path = find_folder(extract_path, "test")

print("\nTrain folder:", train_path)
print("Validation folder:", valid_path)
print("Test folder:", test_path)


IMG_SIZE = (224, 224)
BATCH_SIZE = 16


print("\nLoading training data...")

train_data = tf.keras.utils.image_dataset_from_directory(
    train_path,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="binary",
    shuffle=True
)


print("\nLoading validation data...")

valid_data = tf.keras.utils.image_dataset_from_directory(
    valid_path,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="binary",
    shuffle=False
)


print("\nLoading test data...")

test_data = tf.keras.utils.image_dataset_from_directory(
    test_path,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="binary",
    shuffle=False
)


class_names = train_data.class_names

print("\nClasses:", class_names)


AUTOTUNE = tf.data.AUTOTUNE
```

```
train_data = train_data.prefetch(AUTOTUNE)
valid_data = valid_data.prefetch(AUTOTUNE)
test_data = test_data.prefetch(AUTOTUNE)


print("\nShowing sample images...")


plt.figure(figsize=(10, 8))

for images, labels in train_data.take(1):

    for i in range(min(9, len(images))):

        plt.subplot(3, 3, i + 1)

        plt.imshow(images[i].numpy().astype("uint8"))

        label = int(labels[i].numpy()[0])

        plt.title(class_names[label])

        plt.axis("off")

plt.tight_layout()
plt.show()


data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1)
])


print("\n==============================")
print("        SIMPLE CNN MODEL")
print("==============================")


cnn = models.Sequential([

    layers.Input(shape=(224, 224, 3)),

    data_augmentation,

    layers.Rescaling(1./255),

    layers.Conv2D(
        32,
```

```
        3,
        activation="relu"
    ),

    layers.MaxPooling2D(),

    layers.Conv2D(
        64,
        3,
        activation="relu"
    ),

    layers.MaxPooling2D(),

    layers.Conv2D(
        128,
        3,
        activation="relu"
    ),

    layers.MaxPooling2D(),

    layers.Flatten(),

    layers.Dense(
        128,
        activation="relu"
    ),

    layers.Dropout(0.5),

    layers.Dense(
        1,
        activation="sigmoid"
    )
])


cnn.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)


cnn.summary()


print("\nTraining CNN...")
```

```
history_cnn = cnn.fit(
    train_data,
    validation_data=valid_data,
    epochs=15
)


cnn_loss, cnn_accuracy = cnn.evaluate(
    test_data,
    verbose=0
)


print("\nCNN Test Accuracy:",
      round(cnn_accuracy * 100, 2), "%")


print("\n==============================")
print("        MOBILENETV2 MODEL")
print("==============================")


base_model = MobileNetV2(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)


base_model.trainable = False


mobilenet = models.Sequential([

    layers.Input(shape=(224, 224, 3)),

    layers.Rescaling(
        1./127.5,
        offset=-1
    ),

    base_model,

    layers.GlobalAveragePooling2D(),

    layers.Dropout(0.3),

    layers.Dense(
        1,
        activation="sigmoid"
    )
```

```
])


mobilenet.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)


mobilenet.summary()


print("\nTraining MobileNetV2...")


history_mobilenet = mobilenet.fit(
    train_data,
    validation_data=valid_data,
    epochs=10
)


mobile_loss, mobile_accuracy = mobilenet.evaluate(
    test_data,
    verbose=0
)


print("\nMobileNetV2 Test Accuracy:",
      round(mobile_accuracy * 100, 2), "%")


print("\n==============================")
print("       MODEL COMPARISON")
print("==============================")


print(
    "CNN Accuracy        :",
    round(cnn_accuracy * 100, 2),
    "%"
)

print(
    "MobileNetV2 Accuracy:",
    round(mobile_accuracy * 100, 2),
    "%"
)
```

```
models_list = [
    "CNN",
    "MobileNetV2"
]


accuracy_list = [
    cnn_accuracy * 100,
    mobile_accuracy * 100
]


plt.figure(figsize=(7, 5))

bars = plt.bar(
    models_list,
    accuracy_list
)

plt.ylabel("Accuracy (%)")

plt.xlabel("Model")

plt.title("CNN vs MobileNetV2")

plt.ylim(0, 100)


for bar, value in zip(
    bars,
    accuracy_list
):

    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value + 1,
        f"{value:.2f}%",
        ha="center"
    )


plt.show()


print("\n==============================")
print("      ACCURACY GRAPHS")
print("==============================")


plt.figure(figsize=(8, 5))
```

```
plt.plot(
    history_cnn.history["accuracy"],
    label="CNN Training"
)

plt.plot(
    history_cnn.history["val_accuracy"],
    label="CNN Validation"
)

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.title("CNN Training and Validation Accuracy")

plt.legend()

plt.show()


plt.figure(figsize=(8, 5))

plt.plot(
    history_mobilenet.history["accuracy"],
    label="MobileNetV2 Training"
)

plt.plot(
    history_mobilenet.history["val_accuracy"],
    label="MobileNetV2 Validation"
)

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.title(
    "MobileNetV2 Training and Validation Accuracy"
)

plt.legend()

plt.show()


print("\n==============================")
print("      MOBILENETV2 EVALUATION")
print("==============================")
```

```
y_true = []

y_pred = []


for images, labels in test_data:

    predictions = mobilenet.predict(
        images,
        verbose=0
    )

    predictions = (
        predictions > 0.5
    ).astype(int).flatten()

    y_true.extend(
        labels.numpy().astype(int)
    )

    y_pred.extend(
        predictions
    )


print("\nClassification Report:\n")


print(
    classification_report(
        y_true,
        y_pred,
        target_names=class_names,
        zero_division=0
    )
)


print("\nConfusion Matrix:\n")


cm = confusion_matrix(
    y_true,
    y_pred
)


print(cm)


plt.figure(figsize=(6, 5))
```

```
plt.imshow(cm)

plt.title(
    "MobileNetV2 Confusion Matrix"
)

plt.xlabel(
    "Predicted Label"
)

plt.ylabel(
    "Actual Label"
)


plt.xticks(
    range(len(class_names)),
    class_names
)

plt.yticks(
    range(len(class_names)),
    class_names
)


for i in range(len(cm)):

    for j in range(len(cm)):

        plt.text(
            j,
            i,
            cm[i, j],
            ha="center",
            va="center"
        )


plt.tight_layout()

plt.show()


print("\n==============================")
print("       TEST YOUR OWN IMAGE")
print("==============================")


uploaded = files.upload()
```

```
filename = list(
    uploaded.keys()
)[0]


original_img = load_img(
    filename
)


img = load_img(
    filename,
    target_size=(224, 224)
)


img_array = img_to_array(
    img
)


img_array = np.expand_dims(
    img_array,
    axis=0
)


prediction = mobilenet.predict(
    img_array,
    verbose=0
)[0][0]


if prediction > 0.5:

    result = "ROTTEN"

    confidence = prediction * 100

else:

    result = "FRESH"

    confidence = (1 - prediction) * 100


print("\nImage:", filename)

print(
    "Prediction:",
```

```
    result
)

print(
    "Confidence:",
    round(confidence, 2),
    "%"
)


plt.figure(figsize=(6, 6))

plt.imshow(original_img)

plt.axis("off")

plt.title(
    f"Prediction: {result}\n"
    f"Confidence: {confidence:.2f}%"
)

plt.show()
```

```
TensorFlow Version: 2.21.0

Downloading dataset...
Dataset downloaded successfully
Dataset extracted successfully

Checking dataset folders...
/content/fruit_dataset/Dataset/test/rotten : 12 images
/content/fruit_dataset/Dataset/test/fresh : 12 images
/content/fruit_dataset/Dataset/train/rotten : 142 images
/content/fruit_dataset/Dataset/train/fresh : 145 images
/content/fruit_dataset/Dataset/valid/rotten : 24 images
/content/fruit_dataset/Dataset/valid/fresh : 24 images

Train folder: /content/fruit_dataset/Dataset/train
Validation folder: /content/fruit_dataset/Dataset/valid
Test folder: /content/fruit_dataset/Dataset/test

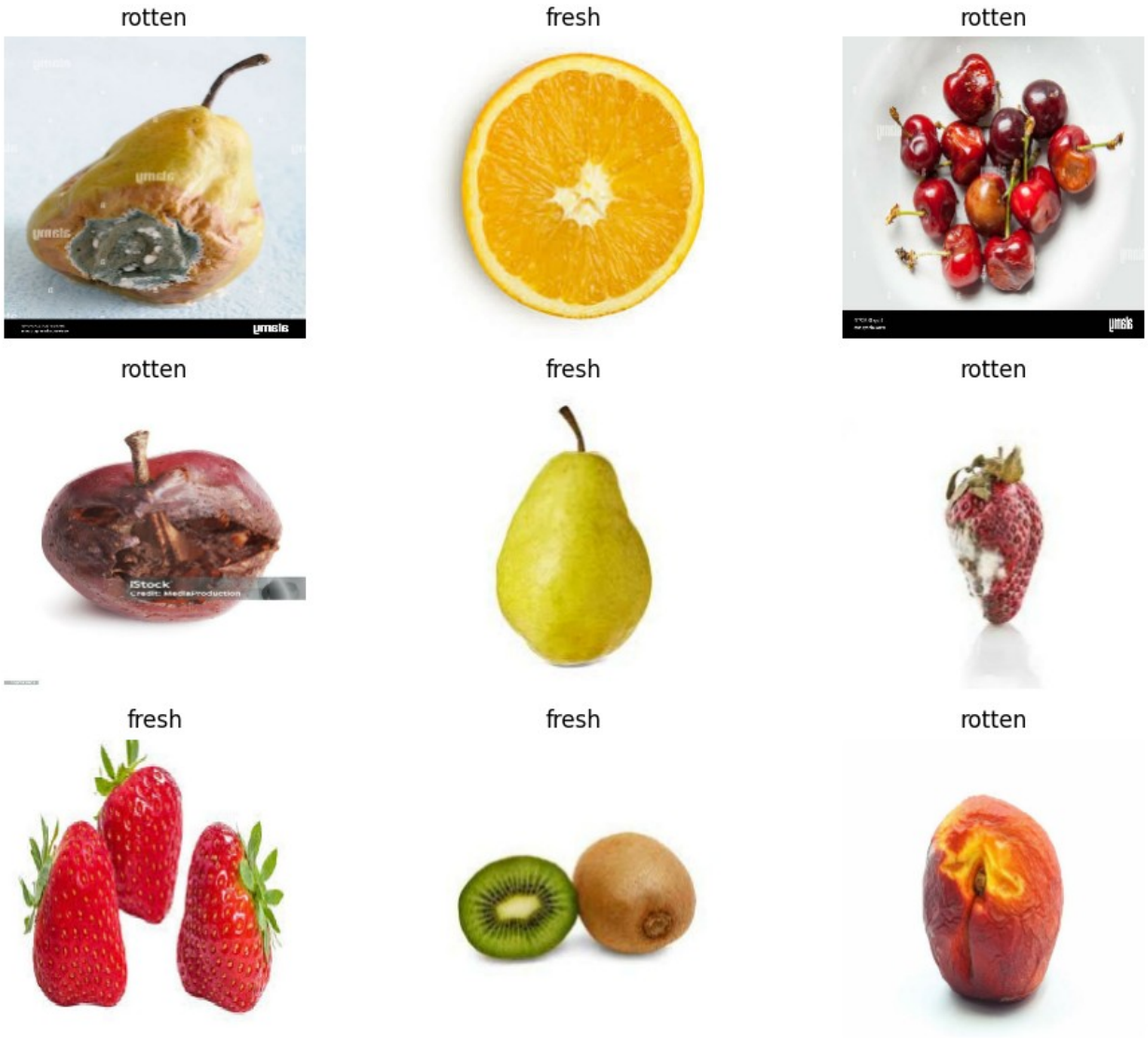
Loading training data...
Found 287 files belonging to 2 classes.

Loading validation data...
Found 48 files belonging to 2 classes.

Loading test data...
Found 24 files belonging to 2 classes.

Classes: ['fresh', 'rotten']
```

```
Showing sample images...
```




```
================================
        SIMPLE CNN MODEL
================================

Model: "sequential_4"
```

Layer (type)	Output Shape	Param #

```
━━━━━━┫
┃ sequential_3 (Sequential)        ┃ (None, 224, 224, 3)     ┃
0 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━
━━━━┫
┃ rescaling_2 (Rescaling)          ┃ (None, 224, 224, 3)     ┃
0 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━
━━━━┫
┃ conv2d_3 (Conv2D)                ┃ (None, 222, 222, 32)    ┃
896 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━
━━━━┫
┃ max_pooling2d_3 (MaxPooling2D)   ┃ (None, 111, 111, 32)    ┃
0 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━
━━━━┫
┃ conv2d_4 (Conv2D)                ┃ (None, 109, 109, 64)    ┃
18,496 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━
━━━━┫
┃ max_pooling2d_4 (MaxPooling2D)   ┃ (None, 54, 54, 64)      ┃
0 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━
━━━━┫
┃ conv2d_5 (Conv2D)                ┃ (None, 52, 52, 128)     ┃
73,856 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━
━━━━┫
┃ max_pooling2d_5 (MaxPooling2D)   ┃ (None, 26, 26, 128)     ┃
0 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━
━━━━┫
┃ flatten_1 (Flatten)              ┃ (None, 86528)           ┃
0 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━
━━━━┫
┃ dense_3 (Dense)                  ┃ (None, 128)             ┃
11,075,712 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━
━━━━┫
┃ dropout_2 (Dropout)              ┃ (None, 128)             ┃
0 ┃
┣━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━━━━━━━━━━━━━━━━━╋━━━━━━━━━
━━━━┫
┃ dense_4 (Dense)                  ┃ (None, 1)               ┃
129 ┃
┗━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━━━━━━━━━━━━━━━━━┻━━━━━━━━━
━━━━┛
```

```
 Total params: 11,169,089 (42.61 MB)

 Trainable params: 11,169,089 (42.61 MB)

 Non-trainable params: 0 (0.00 B)


Training CNN...
Epoch 1/15
18/18 ━━━━━━━━━━━━━━━━━━━━ 5s 93ms/step - accuracy: 0.5226 - loss:
0.9242 - val_accuracy: 0.6458 - val_loss: 0.6481
Epoch 2/15
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 48ms/step - accuracy: 0.6481 - loss:
0.6127 - val_accuracy: 0.6667 - val_loss: 0.5592
Epoch 3/15
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.6620 - loss:
0.6039 - val_accuracy: 0.7083 - val_loss: 0.5694
Epoch 4/15
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.7038 - loss:
0.5385 - val_accuracy: 0.6667 - val_loss: 0.6340
Epoch 5/15
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.7352 - loss:
0.4967 - val_accuracy: 0.6458 - val_loss: 0.6779
Epoch 6/15
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.7352 - loss:
0.5729 - val_accuracy: 0.7083 - val_loss: 0.6862
Epoch 7/15
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.7178 - loss:
0.4903 - val_accuracy: 0.6875 - val_loss: 0.8443
Epoch 8/15
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.7875 - loss:
0.4407 - val_accuracy: 0.6458 - val_loss: 1.0003
Epoch 9/15
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.7596 - loss:
0.4361 - val_accuracy: 0.7083 - val_loss: 1.0710
Epoch 10/15
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.8223 - loss:
0.4332 - val_accuracy: 0.5833 - val_loss: 0.7404
Epoch 11/15
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.7944 - loss:
0.4246 - val_accuracy: 0.7500 - val_loss: 1.1806
Epoch 12/15
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 45ms/step - accuracy: 0.7875 - loss:
0.4023 - val_accuracy: 0.6875 - val_loss: 1.5302
Epoch 13/15
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 68ms/step - accuracy: 0.7944 - loss:
0.3660 - val_accuracy: 0.6667 - val_loss: 1.6397
Epoch 14/15
18/18 ━━━━━━━━━━━━━━━━━━━━ 2s 50ms/step - accuracy: 0.8153 - loss:
0.3443 - val_accuracy: 0.7292 - val_loss: 1.6231
```

```
Epoch 15/15
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.8467 - loss:
0.3147 - val_accuracy: 0.7292 - val_loss: 1.3481

CNN Test Accuracy: 70.83 %

==============================
        MOBILENETV2 MODEL
==============================

Model: "sequential_5"

┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━
━━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━
━━━━┩
│ rescaling_3 (Rescaling)              │ (None, 224, 224, 3)         │
0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────
────┤
│ mobilenetv2_1.00_224                 │ (None, 7, 7, 1280)          │
2,257,984 │
│ (Functional)                         │                             │
│
├──────────────────────────────────────┼─────────────────────────────┼─────────────
────┤
│ global_average_pooling2d_1           │ (None, 1280)                │
0 │
│ (GlobalAveragePooling2D)             │                             │
│
├──────────────────────────────────────┼─────────────────────────────┼─────────────
────┤
│ dropout_3 (Dropout)                  │ (None, 1280)                │
0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────
────┤
│ dense_5 (Dense)                      │ (None, 1)                   │
1,281 │
└──────────────────────────────────────┴─────────────────────────────┴─────────────
────┘

 Total params: 2,259,265 (8.62 MB)

 Trainable params: 1,281 (5.00 KB)

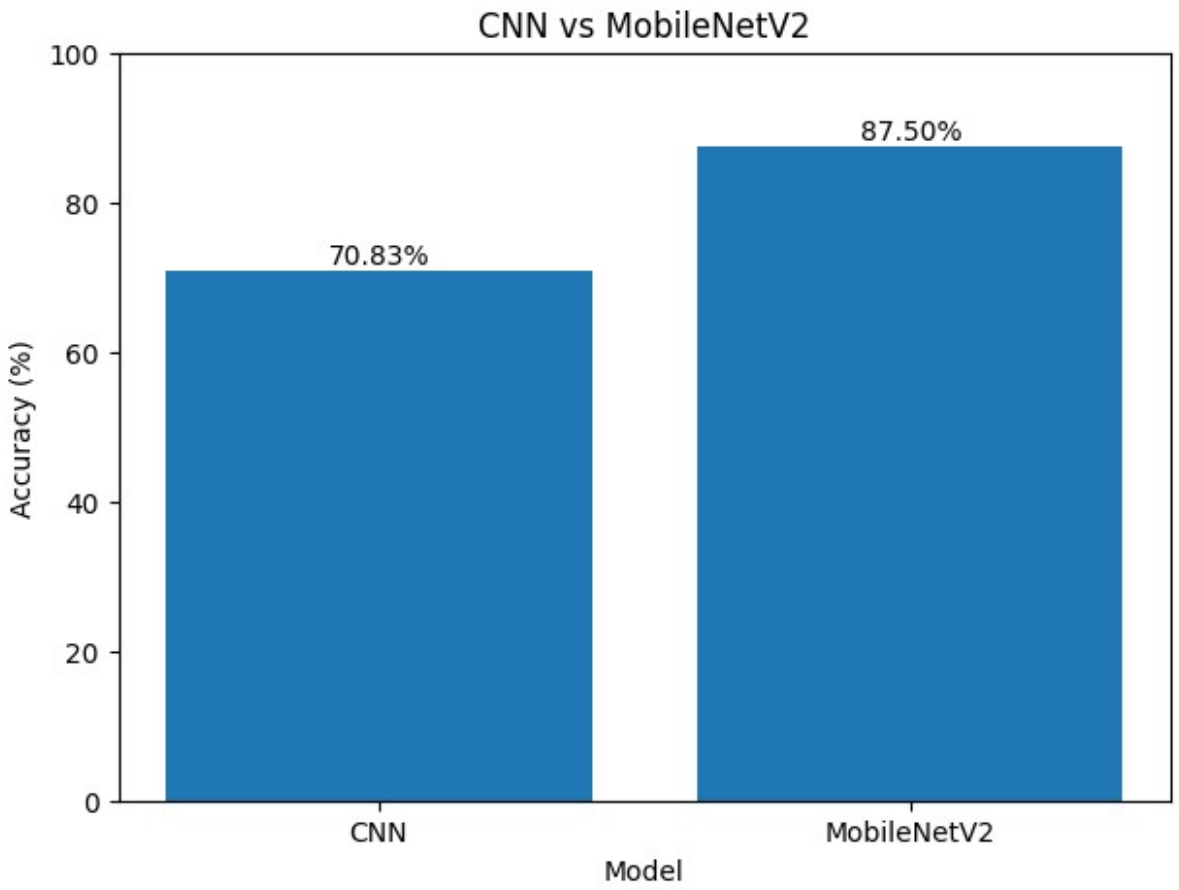
 Non-trainable params: 2,257,984 (8.61 MB)


Training MobileNetV2...
```

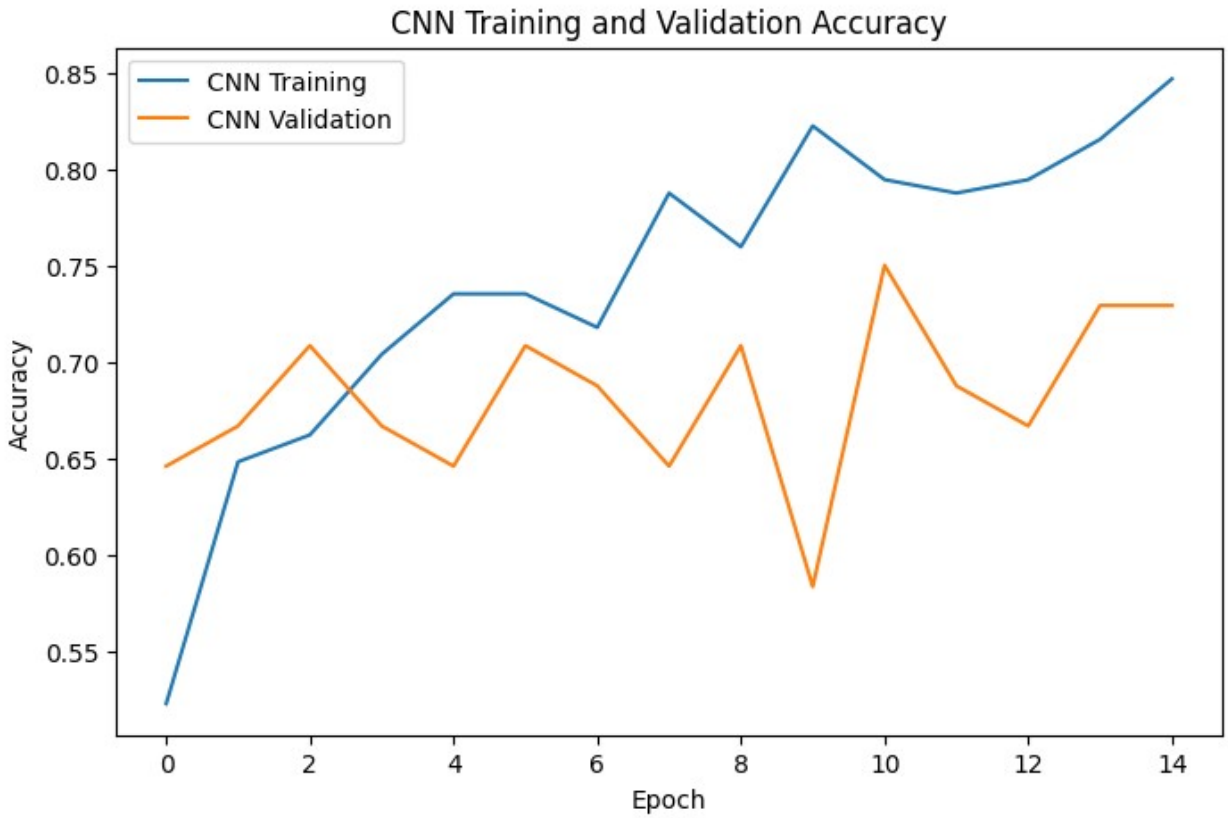
```
Epoch 1/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 15s 454ms/step - accuracy: 0.6202 - loss:
0.6673 - val_accuracy: 0.6875 - val_loss: 0.5373
Epoch 2/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.7561 - loss:
0.4976 - val_accuracy: 0.8125 - val_loss: 0.4681
Epoch 3/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.8467 - loss:
0.3765 - val_accuracy: 0.8750 - val_loss: 0.3857
Epoch 4/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 40ms/step - accuracy: 0.8850 - loss:
0.3103 - val_accuracy: 0.8750 - val_loss: 0.3554
Epoch 5/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 38ms/step - accuracy: 0.9129 - loss:
0.2715 - val_accuracy: 0.8750 - val_loss: 0.3347
Epoch 6/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step - accuracy: 0.9338 - loss:
0.2429 - val_accuracy: 0.8750 - val_loss: 0.3293
Epoch 7/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 2s 71ms/step - accuracy: 0.9443 - loss:
0.2066 - val_accuracy: 0.8750 - val_loss: 0.3277
Epoch 8/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.9617 - loss:
0.1824 - val_accuracy: 0.8750 - val_loss: 0.3148
Epoch 9/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.9582 - loss:
0.1678 - val_accuracy: 0.8750 - val_loss: 0.3051
Epoch 10/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 1s 41ms/step - accuracy: 0.9582 - loss:
0.1649 - val_accuracy: 0.8750 - val_loss: 0.3030

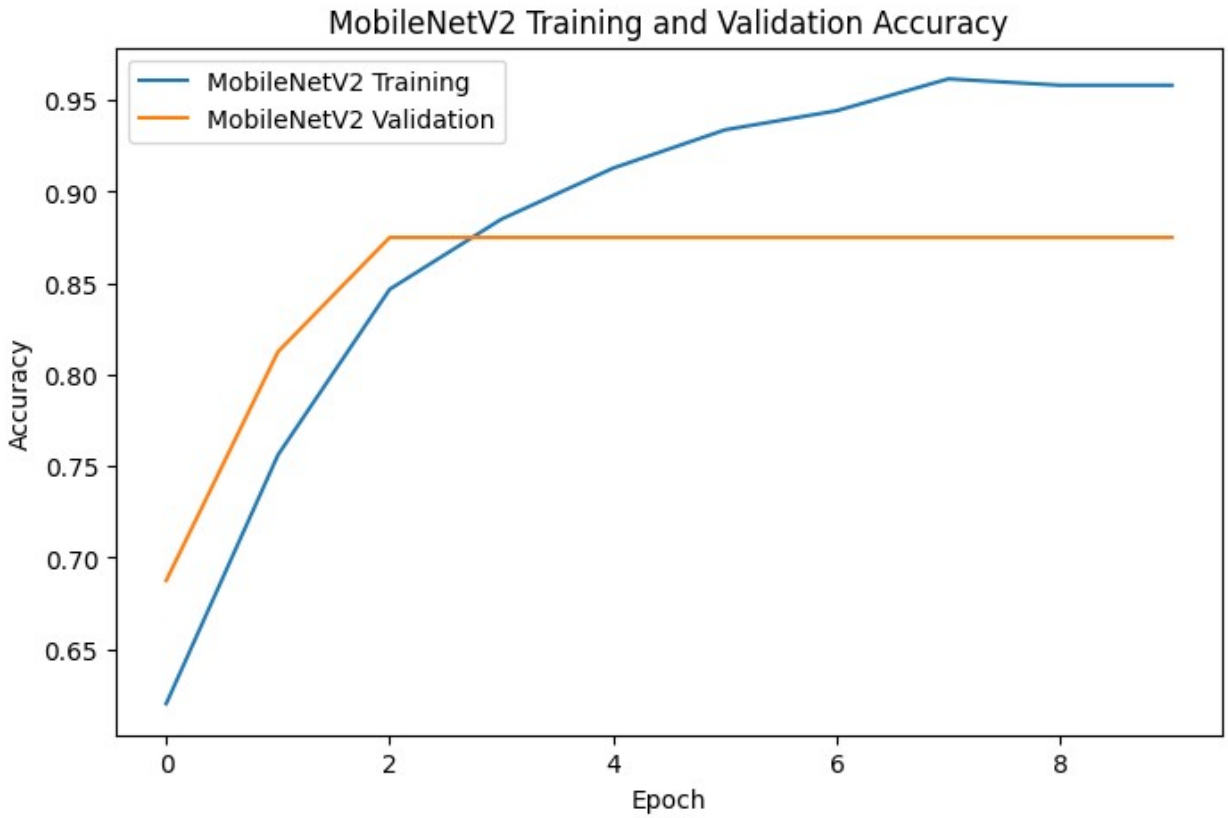
MobileNetV2 Test Accuracy: 87.5 %

==============================
       MODEL COMPARISON
==============================
CNN Accuracy        : 70.83 %
MobileNetV2 Accuracy: 87.5 %
```




```
==============================
      ACCURACY GRAPHS
==============================
```


CNN Training and Validation Accuracy
CNN Training
CNN Validation
Accuracy
Epoch
0.85
0.80
0.75
0.70
0.65
0.60
0.55
0
2
4
6
8
10
12
14


MobileNetV2 Training and Validation Accuracy
MobileNetV2 Training
MobileNetV2 Validation
Accuracy
Epoch
0.95
0.90
0.85
0.80
0.75
0.70
0.65
0
2
4
6
8

```
==============================
      MOBILENETV2 EVALUATION
==============================

Classification Report:

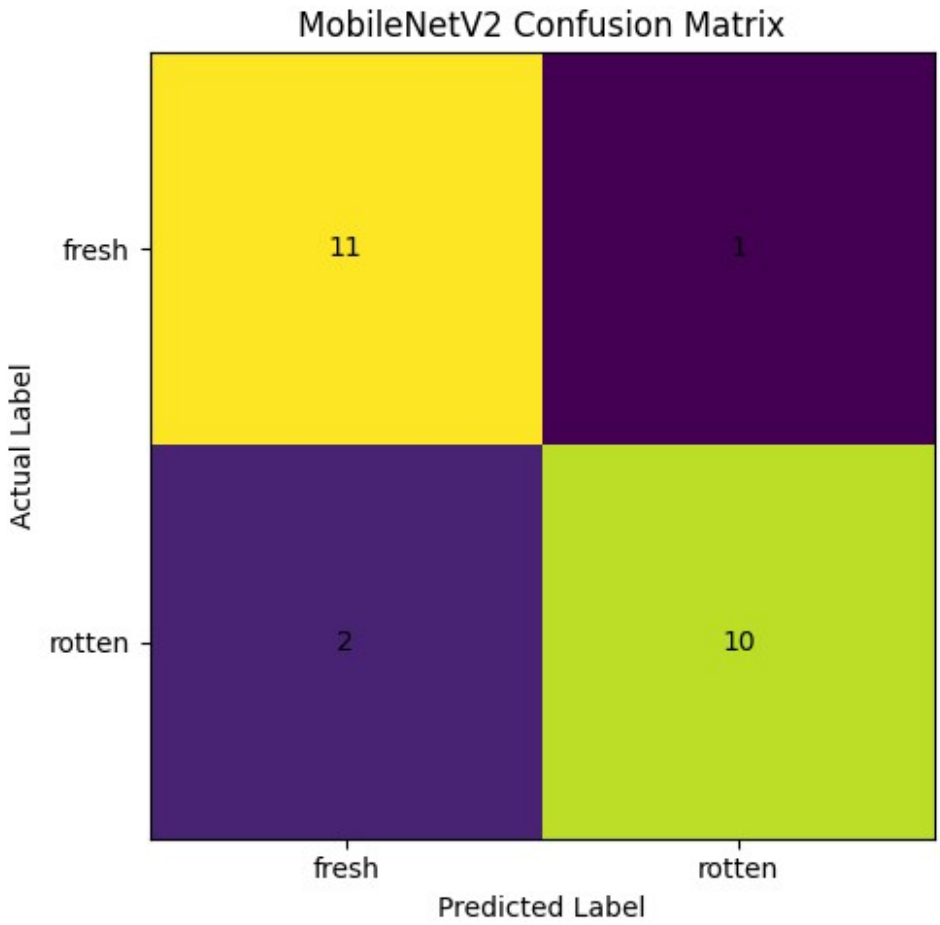
              precision    recall  f1-score   support

       fresh       0.85      0.92      0.88        12
      rotten       0.91      0.83      0.87        12

    accuracy                           0.88        24
   macro avg       0.88      0.88      0.87        24
weighted avg       0.88      0.88      0.87        24


Confusion Matrix:

[[11  1]
 [ 2 10]]
```



```
==============================
       TEST YOUR OWN IMAGE
==============================

<IPython.core.display.HTML object>

Saving Screenshot_7-10-2026_211028_www.bing.com.jpeg to Screenshot_7-
10-2026_211028_www.bing.com (1).jpeg

Image: Screenshot_7-10-2026_211028_www.bing.com (1).jpeg
Prediction: FRESH
Confidence: 88.67 %
```

